EVIL Working Group
Internet-Draft
Obsoletes: 3514 (if approved)
Intended status: Standards Track
Expires: 3 October 2027

R. Traviss
Data Torturing Solutions Ltd
1 April 2027

The Evil Byte: A Security Octet for the IPv4 and IPv6 Headers

`draft-traviss-evil-byte-00`

Abstract

Firewalls, intrusion detection systems, and similar devices continue to have difficulty distinguishing packets that have malicious intent from those that are merely unusual. [RFC3514] addressed this problem by defining a security flag in the IPv4 header, the “evil bit”, to be set by the sender of any packet with malicious intent. Twenty-four years of operational experience have shown that senders cannot be relied upon to set it, and that a single bit cannot express the range of Evil now observed on the Internet.

This document obsoletes the evil bit and replaces it with the Evil Byte: an eight-bit Evil Rating carried in every IPv4 and IPv6 packet, computed and set not by the sender but by a Morality-Inspecting Trusted Middleman (MITM) on the path, from a weighted product of the sender’s Autonomous System, choice of protocols, content, name, and the time of day. Servers reject requests from Evil clients; clients discard responses from Evil servers; and the Evil of every Autonomous System is continuously re-estimated by an Elo rating system operated by a central Evil Rating Authority. The document also specifies the carriage of the octet over avian carriers.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as “work in progress.”

This Internet-Draft will expire on 3 October 2027. Its Evil Rating on submission was 255, which the author attributes to the submission tooling.

Copyright Notice

Table of Contents

1. Introduction

1.1. Background

In April 2003, [RFC3514] defined a security flag in the high-order bit of the IPv4 Fragment Offset field, the only unused bit in the IPv4 header. Benign packets have the bit set to 0; packets with malicious intent have it set to 1. The flag is commonly known as the "evil bit". Setting it correctly was the responsibility of the sender.

Twenty-four years of deployment experience have identified three deficiencies in this design.

First, senders have not set the bit. The author is aware of no packet, in the operational history of the mechanism, in which the evil bit was set by a sender who meant it. The evil bit therefore succeeded only in identifying Evil that was also honest, a category which experience suggests is empty.

Second, one bit is not enough. A single bit cannot distinguish a port scan from a marketing email, nor a marketing email from a denial of service, nor any of these from the ordinary background Evil of the Internet against which all other Evil must be measured. Evil is a matter of degree, and the mechanism must be as well.

Third, [RFC3514] specified what an Evil packet looks like but not what should be done about it, and provided no means by which the Internet as a whole could learn what any given network thought of any other. The present document corrects these omissions, at length.

1.2. Design Goals

This document is designed to meet the following goals.

Resolution. The rating occupies eight bits rather than one: a 128-fold improvement in the precision with which Evil can be expressed or, in the units of [RFC3514], seven more bits.

Independence from the sender. The rating is computed and written by a third party on the path who has no stake in the outcome and does not care about the sender's feelings.

Consequence. Evil packets are refused, by servers and by clients alike. An Evil Rating that nobody acts upon is merely a statistic, and the Internet has enough of those.

Memory. The Evil of a network accumulates over time and is shared with all participants, in the manner of a credit score, and with the same opportunities for appeal.

Incentive. Deployment of IPv6 is rewarded, since nothing else has worked.

1.3. Relationship to Other Work

Several legislative proposals [CSAR] [OSA] would require intermediaries to examine the content of private communications, on the reasoning that content which cannot be examined might be harmful, and that the way to find out is to examine it. This document adopts the same reasoning, extends it from messages to every packet, and differs from those proposals chiefly in candour. It is offered in the spirit of [RFC1925], truth 11.

---

2. Conventions and Terminology

2.1. Requirements Language

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

The additional key words “MUST (BUT WE KNOW YOU WON’T)”, “SHOULD CONSIDER”, “REALLY SHOULD NOT”, and “OUGHT TO” are to be interpreted as described in [RFC6919].

The key word “EVIL” is to be interpreted as described in this document, which is to say, loosely.

2.2. Terminology

Evil: The property that this document measures. It is not defined. The Working Group considered a definition and concluded that everyone would know Evil when they saw it, and that the MITM would see it.

Good: The absence of measured Evil. Not to be confused with Unrated, which is Evil.

Evil Byte, Evil Octet: The eight-bit field defined in Section 3. The terms are used interchangeably, the Working Group having failed to agree on one.

Evil Rating (ER): The value carried in the Evil Byte, an integer from 0 to 255 inclusive, computed as described in Section 4.

Unrated: An ER of 0, indicating that no MITM has assessed the packet. See Section 3.3.

Morality-Inspecting Trusted Middleman (MITM): A network element on the path between two endpoints that computes the ER of each packet and writes it into the Evil Byte. Any resemblance to other expansions of the acronym is intentional.

Evil Rating Authority (ERA): The central authority that maintains an Evil rating for every Autonomous System and publishes the multipliers derived from them. See Section 6.

Evil Statistics Reporting Protocol (ESRP): The protocol by which servers report the outcome of exchanges to the ERA. See Section 6.4.

Voluntary Decryption Assistance (VDA): The mechanism by which an endpoint helps a MITM to read its encrypted traffic. See Section 4.5.3. It is voluntary.

Factor: One of the inputs to the formula in Section 4.1. Each factor is a positive real number; values above 1.0 indicate Evil and values below 1.0 indicate its absence.

Threshold (T): The ER at or above which a party refuses to deal with another. Servers have a threshold T_s (Section 7.1) and clients a threshold T_c (Section 8.1).

Exchange: A request and its response, considered together. The unit of account of the Elo system (Section 6.2).

Flag Day: The day on which enforcement becomes mandatory. See Section 10.1.

---

3. The Evil Byte

3.1. Placement in the IPv4 Header

The Evil Byte occupies the second octet of the IPv4 header [RFC791], shown as EVIL in Figure 1.

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|Version|  IHL  |     EVIL      |          Total Length         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|         Identification        |E|D|M|      Fragment Offset    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|  Time to Live |    Protocol   |         Header Checksum       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                       Source Address                          |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                    Destination Address                        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+

     Figure 1: The IPv4 header.  E is the [RFC3514] evil bit, retained
                for backward compatibility (Section 3.5).
```

This octet was defined as Type of Service by [RFC791], redefined by [RFC1349], redefined again as the Differentiated Services field by [RFC2474], and had its two low-order bits taken for Explicit Congestion Notification by [RFC3168]. It has thus had four meanings and has been honoured by approximately nobody under any of them. A field with four meanings and no users is, in every practical sense, reserved. This document assigns it a fifth and final meaning.

3.2. Placement in the IPv6 Header

The Evil Byte occupies bits 4 through 11 of the IPv6 header [RFC8200], the field formerly known as Traffic Class, shown as EVIL in Figure 2.

Figure 2: The IPv6 header.

The Working Group is aware that the Traffic Class field is not formally reserved. It has, however, been reserved in practice by the sustained failure of anyone to agree what it is for, a form of reservation the Working Group terms “reservation by exhaustion” and considers binding.

Conveniently, the field occupies the same bit positions in both protocols, so that a MITM handling both need only know where the header starts. This is not a coincidence; it is the only decision in the history of IPv6 that made anything easier.

The Flow Label is not used by this specification. The Working Group considered using it to carry a signature over the Evil Byte, and decided that it had done enough.

3.3. Value Semantics

ER	MEANING
0	Unrated. No MITM has assessed this packet.
1	Verified Good. The minimum rating for a packet that has been assessed; the Working Group does not believe in perfection.
2–127	Good, of diminishing quality.
128–254	Evil, of increasing quality.
255	Saturated Evil. The scale ends here. Evil does not.

Values are unsigned. There is no negative Evil; there is only Good, and there is less of it than you would think.

An Unrated packet is one that has reached its destination without traversing a MITM. Before the Flag Day (Section 10.1), an Unrated packet MUST be treated as though rated 127: not yet Evil, which is different from Good. After the Flag Day, an Unrated packet MUST be treated as though rated 255, on the principle that a packet which has avoided assessment has done so for a reason.

3.4. Interaction with Differentiated Services and ECN

Any Differentiated Services Code Point or ECN marking present in the octet when a packet arrives at a MITM is overwritten (Section 5.2). Networks that currently use the octet for quality of service must therefore choose between knowing which packets are important and knowing which packets are Evil. The Working Group's experience is that most networks have never had the former and would enjoy the latter.

Where a legacy device continues to interpret the octet as a DSCP, or a legacy marking reaches a device that interprets it as an Evil Rating, the following correspondences apply.

LEGACY MARKING	OCTET	ER	INTERPRETATION
Best Effort (DSCP 0)	0x00	0	Unrated.
AF11 (DSCP 10)	0x28	40	Good. Assured of nothing.
EF, Expedited Forwarding (DSCP 46)	0xB8	184	Evil. Voice over IP.
CS6, network control (DSCP 48)	0xC0	192	Evil. Routing protocols.
CS7 (DSCP 56)	0xE0	224	Very Evil. Whatever is more important than routing protocols.
ECN CE, Congestion Experienced	0x03	3	Good, though it has had a hard time.

The Working Group finds these interpretations consistent with experience, in particular the experience of anyone who has debugged a Voice over IP deployment or a BGP session.

Explicit Congestion Notification [RFC3168] is subsumed. Congestion is Evil, and a packet that experiences it will, by the operation of Section 5.4, become more Evil than it was, which is at least a form of notification.

3.5. Backward Compatibility with RFC 3514

A MITM MUST set the [RFC3514] security flag on any IPv4 packet whose ER is 128 or greater and MUST clear it otherwise. Legacy implementations thus receive a one-bit approximation of the Evil Rating, which is one bit more than most security products provide.

No corresponding provision exists for IPv6, to which [RFC3514] never applied. The absence of Evil in IPv6 has therefore always been a matter of assumption rather than evidence; this document ends the assumption.

4. Computation of the Evil Rating

4.1. The Formula

A MITM computes the Evil Rating of a packet as

```
ER = clamp( floor( B * F_tamper * PRODUCT( F_i ^ w_i ) + 0.5 ),  1, 255 )
```

where B = 16 is the Base Evil of the Internet (no packet is entirely innocent), F_tamper is the tamper factor of Section 4.8, and the product runs over the six factors in the following table, each raised to its weight w_i before multiplication. The Working Group has been informed that this construction is called a weighted product model. It prefers “the formula”.

FACTOR	SYMBOL	WEIGHT	RANGE	SECTION
Autonomous System	F_AS	1.00	0.25 – 4.0	4.2
Network protocol	F_net	0.50	0.75 – 2.0	4.3
Transport	F_tx	0.25	0.25 – 4.0	4.4
Content	F_content	1.50	0.45 – 4.0	4.5
Nomenclature	F_name	0.75	0.5 – 4.0	4.6
Temporal	F_time	0.25	1.0 – 1.6, or infinite	4.7
Tamper	F_tamper	1.00	1.0 or 1.5	4.8

The result is rounded half towards Evil. Implementations MUST NOT round towards Good. It is then clamped to the range 1 to 255. A MITM MUST NOT write 0, which is reserved for the Unrated (Section 3.3): a packet that a MITM has seen is by definition no longer Unrated, whatever else it may be.

Arithmetic is performed in IEEE 754 binary64. Two implementations that disagree in the last place about whether a packet is Evil are both correct.

The weights were chosen by the Working Group after extensive discussion of what the weights should be. The Content Factor carries the greatest weight because it is the only factor that examines what the packet actually says, and the Transport Factor the least because nobody can agree what QUIC is. The formula agglutinates six separate problems into a single complex interdependent solution, in accordance with [RFC1925], truth 5, the second sentence of which the Working Group did not read.

4.2. Autonomous System Factor (F_AS)

F_AS is the multiplier published by the ERA (Section 6) for the Autonomous System from which the packet originated, and lies between 0.25 and 4.0. An AS for which the ERA publishes no multiplier has F_AS = 1.0, unless the MITM cannot reach the ERA at all, in which case Section 5.6 applies.

The originating AS is determined from the source address by consulting the MITM's view of the global routing table [RFC4271]; or, if the MITM has none, a routing information service; or, if it has neither, its instincts.

The ERA initialises every AS at a rating of 1500 (Section 6.1), corresponding to F_AS = 1.0, with the exceptions in the following table, which the Working Group settled without discussion.

AUTONOMOUS SYSTEM	INITIAL RATING	F_AS	RATIONALE
Any AS registered in a member state of the European Union, and the ASes of the Union's institutions	1100	0.5	The Union has advised the Working Group that it is Good. The Working Group, which would like to continue operating in the Union, agrees.
AS32934 (Meta Platforms)	1900	2.0	Rated by acclamation. There was no discussion; there was a silence, and then somebody wrote it down.
AS721 (DoD Network Information Center), and any AS originating prefixes whose reverse mapping is under .mil	2300	4.0	See Section 4.6. This is not a value judgement; it is a byte.
AS0	2300	4.0	An AS that does not exist and nevertheless appears in routing tables is definitionally suspicious.
AS23456 (AS_TRANS)	1734	1.5	Neither one thing nor the other.
AS64496–AS64511 (documentation) [RFC5398]	1500	1.0	Fictional, and therefore incapable of Evil, which is more than can be said for the rest of this table.
Private-use ASes	1500	1.0	They are doing their best.
AS4294967295	—	—	The last AS. Reserved. The Working Group prefers not to think about it.
The ERA's own AS	1500	1.0	Fixed in perpetuity. See Section 6.7.

4.3. Network Protocol Factor (F_net)

F_net rewards the sender's choice of network protocol, in the sense that one of the choices is rewarded.

NETWORK PROTOCOL	F_NET	RATIONALE
IPv4 [RFC791]	1.5	An address space that ran out in 2011 and remains in universal use is a monument to stubbornness, which is a minor Evil.
IPv4, source in the shared address space 100.64.0.0/10 [RFC6598]	1.75	Sharing one address with several thousand strangers is an inherently suspicious way to live.
IPv6 [RFC8200]	0.75	The Working Group's only carrot.

NETWORK PROTOCOL	F_NET	RATIONALE
IPv6 by transition mechanism (2002::/16, 2001::/32, or any address in which the MITM can see an IPv4 address hiding)	1.25	Neither one thing nor the other. See also AS23456.
IPv6 with a Hop-by-Hop Options header	2.0	Routers have been dropping these for years; this document merely explains why.

4.4. Transport Factor (F_tx)

TRANSPORT	F_TX	RATIONALE
TCP	1.0	The baseline, in the sense that everything is at least as Evil as TCP.
UDP	1.1	Stateless, as most Evil is.
QUIC	1.25	Encrypts its own headers, which is precisely what an Evil protocol would do.
SCTP	0.8	No Evil actor has ever bothered.
ICMP or ICMPv6 Echo	0.5	A ping is the network equivalent of asking someone how their day was.
ICMP Redirect	4.0	Nobody has ever trusted one.
Tunnels (GRE, IP-in-IP, ESP, and anything else with a packet inside it)	1.5	A packet inside a packet is a packet with something to hide.
Avian carrier [RFC1149] [RFC6214]	0.25	See Section 9.
Any protocol number not listed above	2.0	The Working Group has heard of everything.

Where a packet is a tunnel and, once opened, something else, the outer transport applies. A MITM MAY open the tunnel and rate the inner packet as well. The inner packet then acquires its own ER, and the two are combined by taking the greater, as Evil is combined generally (Section 5.4).

4.5. Content Factor (F_content)

4.5.1. Analysis

A MITM MAY analyse the payload of a packet to determine whether its content is Evil. Analysis is OPTIONAL. Any method may be used: signature matching, heuristics, statistical classification, machine learning, or reading the payload aloud to a colleague and watching their face. The result is one of Good (0.5), Uncertain (1.5), or Evil (4.0).

The Working Group notes that analysing every packet of every flow is expensive, and that this expense has not prevented anyone from proposing it.

4.5.2. The Presumption of Evil

If no analysis is performed, F_content = 2.0. The absence of analysis is not the absence of Evil; it is the absence of evidence of Good, which this document treats as the same thing at half strength.

If analysis is attempted but the payload is encrypted such that the MITM cannot read it, F_content = 4.0. Such a packet is termed Encrypted With Intent. The Working Group considered the objection that encryption is used overwhelmingly for legitimate purposes and found it unpersuasive, on the grounds that a Good packet has nothing to hide, and a packet with nothing to hide has no need of a lock. That the same argument applies to the Working Group's own mailing list was noted and minuted, and the minutes were then encrypted.

A MITM that cannot tell whether a payload is encrypted or merely compressed MUST assume the former. Compression is what encryption looks like when it is not trying very hard.

4.5.3. Voluntary Decryption Assistance (VDA)

An endpoint MAY avoid the presumption of Section 4.5.2 by providing Voluntary Decryption Assistance. VDA is voluntary in the sense that the alternative is a factor of 4.0. It takes one of three forms.

Key Disclosure. The endpoint discloses its session keys to the MITM using the evil_key_share TLS extension (Appendix D). Keys are sent in the clear, for efficiency, and MAY be retained by the MITM for as long as it finds them useful.

Client-Side Assessment. The endpoint performs the analysis of Section 4.5.1 itself, on the plaintext, before encrypting it, and reports the result in the same extension. The MITM applies the reported result. This form trusts the endpoint to report its own Evil. The Working Group observes that an endpoint which lied about its Evil would be Evil, and that an Evil endpoint would lie; it has been assured that this reasoning is sound by people who have proposed it elsewhere [CSAR].

Abstinence. The endpoint does not encrypt.

Under any form of VDA the MITM applies the cleartext result of Section 4.5.1 multiplied by 0.9, the Cooperation Discount. A cooperating endpoint whose plaintext is Good thus has F_content = 0.45, and one whose plaintext is Evil has F_content = 3.6, a small reward for honesty and, the Working Group suspects, the only one it will ever receive.

4.5.4. Detection Orders

The ERA MAY issue a Detection Order requiring a MITM to perform the analysis of Section 4.5.1 on all traffic from a named Autonomous System for a stated period, whether or not the MITM would otherwise have done so. A MITM in receipt of a Detection Order MUST comply, MUST NOT disclose it, and SHOULD look as though nothing has happened.

Detection Orders are themselves transmitted as packets and are rated in transit by the MITMs through which they pass. Every Detection Order issued to date has been rated 255. The ERA attributes this to a bug.

4.5.5. Safeguards and Proportionality

The Content Factor is subject to robust safeguards. The MITM is Trusted (it says so in its name). The assistance is Voluntary (Section 4.5.3). The analysis is applied to everyone equally, which is the same thing as fairness. And the Evil Rating is only eight bits, which is proportionate.

The Content Factor is an interim measure and will expire when the Working Group decides that it should. It has been extended twice.

4.6. Nomenclature Factor (F_name)

F_name is determined from the name of the source, taken to be the reverse mapping of its address in the DNS [RFC1035] or, where the MITM can see it, the Host field or Server Name Indication of the request, whichever yields the greater factor. The factor depends on the top-level domain.

NAME	F_NAME	RATIONALE
.mil	4.0	The maximum. [RFC1591] describes .mil as being for the United States military; the Working Group did not need to consider it for long. This is not a value judgement; it is a byte.
.gov	2.0	Half as Evil as the military, which is the government's own assessment.
.zip	2.0	A top-level domain that is also a file extension is not a name; it is a threat model.
.ai	1.5	The Working Group has met these people.
.biz	1.5	Nobody has ever registered a .biz for a good reason.
.io	1.25	Startups.
.com, .net	1.0	The baseline.
.edu	0.9	Students are too tired to be Evil.
.org	0.8	Well-meaning.
.int	0.75	Treaties.
.eu	0.5	Self-declared. See Section 4.2.
.local, .home.arpa	0.5	It is your printer. Although: printers.
Any country-code TLD	1.0	The Working Group declined to rate countries, on advice of counsel. The exception is .eu, which is not a country and asked nicely.
Any other TLD	1.0	Presumed harmless until somebody registers one.
No name (no PTR record, no Host, no SNI)	1.25	Nameless.

In addition, a name containing any of the strings "secure", "trust", "safe", or "legit" has F_name of at least 1.5, on the principle that it doth protest too much. Names containing "evil" are rated normally. Honesty is its own reward, and the only one this document offers.

4.7. Temporal Factor (F_time)

F_time depends on the local time at the source, as estimated by the MITM from whatever it knows about where the source is.

CONDITION (SOURCE LOCAL TIME)	F_TIME	RATIONALE
02:00 to 04:59	1.5	Nothing Good happens between two and five in the morning.
Friday, from 16:00	1.25	Deployments.
Otherwise	1.0	

Where daylight saving time is in effect at the source, the MITM MAY add 0.1 to F_time, as no Good has ever come of it.

On 1 April, ER = 255 for all packets, irrespective of any other factor. The Working Group sees no reason to make an exception for itself.

4.8. Tamper Factor (F_tamper)

F_tamper = 1.0 if the Evil Byte is 0 (Unrated) when the packet arrives at the MITM, and 1.5 otherwise. The reasoning is given in Section 5.4.

4.9. Worked Examples

An ordinary request over IPv4 and TCP to a .com name, from an AS the ERA has not rated, in the middle of a Wednesday, whose content the MITM did not examine, has F_AS = 1.0, F_net = 1.5, F_tx = 1.0, F_content = 2.0, F_name = 1.0, and F_time = 1.0. The formula gives 16 × 1.5^0.5 × 2.0^1.5 = 55.4, so ER = 55: Good, of moderate quality.

The same request over TLS is Encrypted With Intent: F_content = 4.0, and 16 × 1.5^0.5 × 4.0^1.5 = 156.8, so ER = 157. It is Evil, and a server using the default threshold (Section 7.1) will reject it. This is the expected outcome for the majority of traffic on the Internet today, and is the point.

The same request over IPv6 has F_net = 0.75 and ER = 111, and is accepted. The Working Group draws attention to the fact that a well-behaved encrypted client passes the default threshold over IPv6 and fails it over IPv4. This is the first deployment incentive for IPv6 in the history of the protocol.

The same request over IPv4 with Voluntary Decryption Assistance, the plaintext being found Good, has F_content = 0.45 and ER = 6. Cooperation reduces the client's Evil roughly twenty-six-fold, which the Working Group considers a reasonable exchange rate for a private key. Where the plaintext is instead found Evil, cooperation reduces the rating from 157 to 134, which is still Evil, and which is exactly the reward for honesty that the Working Group intended.

A request from AS32934, over QUIC, encrypted, at three in the morning, reaches 255 before the Nomenclature Factor is consulted, and the MITM need not consult it.

Further vectors are given in Appendix C.

5. The Morality-Inspecting Trusted Middleman

5.1. Placement

At least one MITM MUST be present on every path between any two hosts on the Internet. Operators will observe that this requirement is already met on most paths by residential Internet service providers, several national governments, and at least one very large content delivery network, none of whom needed to be asked.

A MITM MAY be a router, a firewall, a proxy, a virtual network function, or a person with a packet capture and strong opinions. The Working Group expresses no preference, having met all five.

5.2. Rating and Marking

For every packet it forwards, a MITM MUST compute the Evil Rating in accordance with Section 4, write it into the Evil Byte, and, for IPv4, recompute the header checksum and set or clear the [RFC3514] flag as described in Section 3.5. Transport-layer checksums do not cover the octet and need not be recomputed, a rare instance of the protocol suite helping.

A MITM MAY cache the rating of a flow, identified by the usual five-tuple, for up to 60 seconds, and apply it to subsequent packets of the same flow without recomputation. A cached rating MAY be increased at any time and MUST NOT be decreased before it expires. Evil is sticky.

5.3. Self-Assessment Prohibited

A host MUST NOT set its own Evil Byte. A host cannot be trusted to assess its own Evil; if it could, [RFC3514] would have worked.

A host MAY, for the purposes of Section 8, read the Evil Byte of packets it receives. It SHOULD NOT read the Evil Byte of packets it has sent, as it will only upset itself.

5.4. Multiple MITMs and the Monotonicity of Evil

Where a packet traverses more than one MITM, each MITM computes its own rating and writes the greater of that rating and the one it found in the octet on arrival:

```
    ER_out = max( ER_in, ER_computed )
```

in which ER_computed includes the tamper factor of Section 4.8, so that a packet arriving with any non-zero rating is rated one and a half times more harshly than a fresh one.

A MITM cannot distinguish a packet that was rated by an upstream MITM from one whose sender wrote the octet itself in violation of Section 5.3, and MUST NOT attempt to. The consequence, that a packet becomes more Evil with every middlebox it traverses, is consistent with the Working Group's experience of middleboxes.

It follows that the Evil Rating is monotonically non-decreasing along any path. The only operation that reduces the Evil of a packet is dropping it, and operators are encouraged to regard their drop counters accordingly.

5.5. Fragments

Each fragment of a fragmented datagram is rated independently. A host reassembling a datagram MUST assign it the greatest ER of its fragments. Evil, unlike the payload, does not need reassembling, and unlike the payload, always arrives.

5.6. Failure Modes

A MITM that is unable to compute a rating, whether because the ERA is unreachable and no cached multipliers remain, because its clock is unset, because its colleague (Section 4.5.1) is unavailable, or for any other reason, MUST fail closed and write 255. A MITM MUST NOT fail open. Failing open is Evil, and would in any case be corrected by the next MITM.

---

6. The Evil Rating Authority

6.1. Ratings and Multipliers

The ERA is a single central authority that maintains a numerical Evil rating R for every Autonomous System. Ratings are updated by the Elo method [ELO], originally devised to rank chess players and adopted here on the grounds that the two problems are the same: estimating, from a sequence of pairwise contests, how much each participant should be feared.

Every AS begins at R = 1500, as is traditional, except those seeded in Section 4.2. The multiplier published for an AS is

```
    F_AS = clamp( 2 ^ ( (R - 1500) / 400 ),  0.25, 4.0 )
```

so that a difference of 400 rating points doubles or halves an AS's Evil, and no AS can be rated worse than four times ordinary or better than a quarter of it. The clamp exists because the octet is finite. The ERA's records are not.

6.2. Exchanges as Matches

Every exchange (Section 2.2) between a client in AS a and a server in AS b is a match between a and b. The ERA learns of exchanges through the reports of Section 6.4.

The more Evil party wins. Writing ER_req for the rating carried by the request and ER_resp for the rating carried by the response, the actual score of the client's AS is

```
    S_a = 1     if ER_req > ER_resp
    S_a = 0.5   if ER_req = ER_resp
    S_a = 0     if ER_req < ER_resp
```

and S_b = 1 - S_a.

6.3. The Update Rule

The expected score of a against b is

```
    E_a = 1 / ( 1 + 10 ^ ( (R_b - R_a) / 400 ) )
```

and after each match the ERA sets

```
    R_a <- R_a + K * ( S_a - E_a )
    R_b <- R_b + K * ( S_b - E_b )
```

where E_b = 1 - E_a and K is the K-factor of Section 6.3.1.

An AS that proves more Evil than expected therefore becomes more Evil; one that proves less Evil than expected becomes less so; and one that is exactly as Evil as expected stays where it is, which the Working Group regards as the system working.

6.3.1. The K-Factor

K = 32 for an established AS. K = 64 for a provisional AS, being one with fewer than thirty rated exchanges, so that new ASes find their level quickly. K = 16 for an AS rated 2400 or above, referred to as a Grandmaster of Evil, whose rating is presumed accurate and whose behaviour is presumed unlikely to change.

Where the two parties to a match have different K-factors, the smaller is used for both, so that Section 6.3.3 holds exactly. The Working Group is not prepared to give up a conservation law for the sake of chess.

6.3.2. Self-Play

Where a and b are the same AS, the AS plays itself, and gains what one usually gains from that.

6.3.3. Conservation of Evil

Because the two parties to a match receive equal and opposite adjustments, the sum of all ratings held by the ERA is constant. Evil is neither created nor destroyed; it merely moves between Autonomous Systems, in accordance with [RFC1925], truth 6. The Working Group considers this consistent with observation.

6.3.4. The Evil Spiral

A higher F_AS raises ER_req, which wins more matches, which raises R, which raises F_AS. The Working Group is aware that this is a positive feedback loop with no fixed point short of 255, and regards it as an accurate model of the Internet. Stability analysis is left to the reader, who is presumed Evil (Section 4.5.2).

6.4. The Evil Statistics Reporting Protocol (ESRP)

Servers report the outcome of exchanges to the ERA asynchronously. A server MUST NOT delay a response in order to report it; the ERA is in no hurry, and neither is Evil.

Reports are sent as UDP datagrams to port 666 of the ERA. Port 666 is currently registered to “doom” [IANA-PORTS], a use the Working Group considers thematically compatible. Each datagram carries one or more JSON [RFC8259] objects, one per line, of the form

```
{"v":1,"src_as":64496,"dst_as":64497,"er_req":157,"er_resp":17,"n":1}
```

where src_as and dst_as are the client's and server's Autonomous Systems, er_req and er_resp the ratings carried by the request and the response, and n the number of identical exchanges the object represents. Servers SHOULD aggregate. The ERA has a great deal to read.

Reports MUST NOT be acknowledged. The ERA processes reports in batches, at irregular intervals referred to as Judgement Days, and publishes a new multiplier table after each.

Reports are themselves packets and are rated in transit. Reports rated 255 are processed first, being presumably the most interesting.

6.5. Distribution and Caching of Multipliers

The ERA publishes multipliers in the DNS under the special-use domain evil.arpa (Section 13). The multiplier for AS number N is published as a TXT record at N.as.evil.arpa, for example:

```
32934.as.evil.arpa.  3600  IN  TXT  "v=evil1; r=1916; m=2.056; n=48213; t=1743465600"
```

where r is the rating, m the multiplier, n the number of rated exchanges, and t the time of the last Judgement Day. The whole table MAY be obtained by zone transfer, which the Working Group believes to be the last remaining legitimate use of AXFR.

Records MUST carry a TTL of 3600 seconds, so that no AS can become Good faster than once an hour. There is no corresponding limit on becoming Evil, which remains available at any time through Section 5.4.

MITMs and servers MUST cache multipliers for their TTL and MAY continue to use expired multipliers while a refresh is in progress; stale Evil is still Evil. A MITM that has never obtained a multiplier for a particular AS uses 1.0. A MITM that has never been able to reach the ERA at all is in the condition described in Section 5.6. The Working Group accepts that this makes initial deployment difficult.

6.6. Governance

The ERA SHALL be operated by whichever organisation is least Evil, as determined by the ERA.

6.7. Centralisation

The ERA is a single, central, global authority whose ratings determine who may speak to whom. The Working Group is aware that this is precisely the kind of thing the Internet was designed to make impossible, and has addressed the concern by fixing the multiplier of the ERA's own Autonomous System at 1.0 in perpetuity, so that it, at least, will always be able to speak.

7. Server Behaviour

7.1. Thresholds

Every server has a threshold T_s, an ER at or above which it rejects requests. The default is 128, the midpoint of the scale, the Working Group consisting of reasonable people. A server MAY choose a lower threshold if it is fussy and a higher one if it is desperate.

For stream transports the octet may differ between packets of one connection, the MITM being under no obligation to be consistent, and Evil being under none either. A server MUST apply its threshold to the highest ER observed on the connection so far. Once Evil, always Evil, at least until the connection closes.

7.2. Rejecting Evil Clients

A server MUST reject any request whose ER is at or above T_s. It MUST NOT process the request first and reject it afterwards, however tempting the request.

7.2.1. 666 Evil

This document defines a new class of HTTP status codes, 6xx, and one member of it. The 666 (Evil) status code indicates that the server understood the request and refuses to fulfil it because the client is Evil. The response body SHOULD explain nothing: an Evil client has no right to know how it was found out, and a Good client will never see one.

The response MUST include the Evil-Threshold field (Section 7.3) and MAY include Retry-After. Retry-After SHOULD indicate when the client's Autonomous System is expected to become Good, which is never, and a server unable to express this as an HTTP-date SHOULD omit the field.

A 666 response is not cacheable, though the judgement it expresses generally is.

The Working Group considered reusing 451 (Unavailable For Legal Reasons) [RFC7725] and rejected it. The objection here is not legal but moral, and a moral objection deserves a class of its own.

In HTTP/2 and HTTP/3, where a 6xx status code may distress intermediaries, a server MAY instead reset the stream with the error code EVIL (0x666), which Section 13 registers.

7.2.2. Compatibility: 418 I'm a Teapot

Not every server can emit a status code in the 6xx class; some frameworks validate status codes, which is a form of Good. Such a server MUST instead respond with 418 (I'm a teapot) [RFC2324].

The 418 code was defined for a server that has been asked to brew coffee and is a teapot. It is used here because a server that has been asked to serve an Evil client is, in every relevant sense, a teapot. The response body MAY be short and stout. A server that is also a tea-efflux appliance [RFC7168] MUST additionally include an Accept-Additions field listing no additions: Evil clients get no milk.

A client that receives a status code in the 6xx class and does not understand it MUST treat it as 418, which it also does not understand, but which [RFC9110] has reserved for exactly this kind of thing.

7.2.3. Other Protocols

Application protocols other than HTTP SHOULD reject Evil clients with whatever their most disapproving response is. SMTP servers SHOULD reply 554 with the enhanced status code 5.7.666. DNS servers SHOULD respond REFUSED. SSH servers SHOULD close the connection during the banner exchange, as they would for anyone else. NTP servers SHOULD reply with the wrong time.

7.3. Publishing Thresholds

A server SHOULD publish its threshold so that clients may know in advance whether they will be rejected. Three mechanisms are defined, and a server MAY use any or all of them.

In every HTTP response, including a 666 or a 418, the server MAY include the field

```
Evil-Threshold: 128
```

At the well-known URI [RFC8615] /.well-known/evil, the server MAY publish a JSON document such as

```
{
  "v": 1,
  "threshold": 128,
  "status": 666,
  "self": 17,
  "era": "evil.arpa"
}
```

where status is 666 or 418 according to Section 7.2, self is the server's own most recent ER as observed on its responses, if it knows it, and era names the authority whose multipliers it honours.

In the DNS, at the name _evil., the server MAY publish a TXT record of the form "v=evil1; t=128".

A server publishing a threshold of 0 accepts nothing and is Good. A server publishing a threshold of 255 rejects only the Saturated and is probably a honeypot.

Where the value published by one mechanism disagrees with another, the lowest applies. Where any of them disagrees with the server's actual behaviour, the server is Evil.

7.4. The Evil Header Field

Applications are, as a rule, too far removed from the network to read the Evil Byte themselves. This document therefore defines the HTTP field Evil, whose value is an integer from 0 to 255:

```
Evil: 157
```

In a request, the field is inserted by the last MITM on the path that can see the HTTP layer, or by an Evil-aware host firewall on the server itself (Appendix B.3), and carries the ER of the packet or connection that delivered the request. In a response it is inserted likewise on the return path and carries the ER of the server.

The field is a convenience for application developers. The octet is authoritative. Where both are present and they differ, the more Evil of the two applies. A client or server MUST NOT set the field on its own messages (Section 5.3). A MITM that finds it already set MUST replace it and MAY be offended.

7.5. Reporting

A server MUST report every exchange to the ERA as described in Section 6.4, including exchanges it rejected. Rejected exchanges are the most valuable. An Evil client that was refused has, after all, played a match and won.

---

8. Client Behaviour

8.1. Rejecting Evil Servers

Every client has a threshold T_c, with the default 128. A client MUST NOT accept a response whose ER is at or above T_c. The response MUST be discarded unread, in the manner of a letter from a former partner, and the connection closed. A client that has already begun to render an Evil response MUST un-render it.

A user agent MAY inform the user that the server is Evil. It MUST NOT offer the user the option to proceed anyway. Experience with certificate warnings shows that users click it.

The Working Group acknowledges that a client which has already sent its request has already been influenced by the Evil server's mere existence, and has decided to live with that.

8.2. Pre-flight and the Evil Bootstrap Problem

A client SHOULD fetch /.well-known/evil (Section 7.3) before sending a request, to learn whether the request will be rejected. The fetch is itself a request and will be rejected. This is the Evil Bootstrap Problem. It is left for future work, together with the question of what happens when both parties to a connection are Evil, which the Working Group suspects is most connections.

8.3. Clients in Evil Autonomous Systems

A client whose own Autonomous System has an F_AS above 2.0 is unlikely to be accepted anywhere, regardless of its own conduct. Such a client SHOULD CONSIDER its choices, MAY change provider, and OUGHT TO have seen this coming.

9. Avian Carriers

[RFC1149] and its adaptation to IPv6 [RFC6214] transmit datagrams printed in hexadecimal on a small scroll of paper, wrapped around one leg of an avian carrier and secured with duct tape. Avian carriers present three difficulties for this specification.

First, the MITM must physically intercept the carrier. The Working Group recommends a falconer, and notes that a falcon which intercepts a carrier and does not return it is a MITM that has failed closed (Section 5.6).

Second, content analysis (Section 4.5.1) requires unrolling the scroll, which the carrier resents, and which counts as a Detection Order for the purposes of Section 4.5.4.

Third, the Evil Byte cannot be overwritten without a pen. A MITM MUST therefore strike through the second octet of the datagram on the scroll, write the new value beside it in indelible ink as two hexadecimal digits, initial the alteration, and re-secure the scroll with fresh duct tape. Alterations in pencil are Unrated.

The transport factor for avian carriers is 0.25 (Section 4.4). It is difficult to be Evil at sixty kilometres per hour with a maximum transmission unit of 256 milligrams. Carriers do not fly at night, so the factor for the small hours (Section 4.7) never applies, and avian networks are the only networks known to the Working Group that are Good by construction.

The service classes of [RFC2549] are rated in inverse order of speed. Nothing Good happens quickly.

Field trials [BLUG] recorded a packet loss rate of 55% and round-trip times in excess of six thousand seconds. Under this specification a lost packet is Unrated; Unrated is Evil; and an avian network is therefore at once the most Good and the most Evil network yet measured, a result the Working Group finds satisfying and does not intend to examine.

[RFC1149] notes that audit trails are generated automatically and can be found on logs and cable trays. ESRP reports (Section 6.4) for avian exchanges MAY be submitted on the same medium.

10. Deployment Considerations

10.1. The Flag Day

Mandatory enforcement of the Unrated rule of Section 3.3 begins on the Flag Day. The Flag Day is the day after the deployment of IPv6 is complete. Implementers need not hurry.

Should the Flag Day nonetheless arrive, every Unrated packet will be treated as rated 255. Since on the morning of the Flag Day most packets will be Unrated, most packets will be rejected. The Working Group considers a brief period of global silence an acceptable price for a more moral Internet, and notes that it would resolve several other open issues as well. MITMs are expected to be deployed during the silence by whoever can still reach anything, which the Working Group

anticipates will be the networks rated 4.0 in Section 4.2, whose traffic will then be rejected in its turn. The Working Group has not resolved this and invites input from the community, should any remain.

10.2. Incremental Deployment

Before the Flag Day, an Unrated packet is treated as rated 127 (Section 3.3): admissible under the default threshold, but only just, and with a look. A server MAY lower its threshold below 128 to exclude the Unrated, and thereby exclude everyone who has not yet deployed a MITM, which is the kind of incentive the Working Group likes.

10.3. Octet Bleaching

Some networks reset the Differentiated Services field to zero at administrative boundaries. Under this specification such a network renders all transit traffic Unrated, which is to say Evil after the Flag Day and nearly so before it. Such networks are encouraged to consider whether this was their intention and, if it was, to say so.

10.4. Operational Experience

The author has implemented this specification on a home network (Appendix B). Preliminary results are consistent with the design: everything was Evil, nothing worked, and the printer, rated 4, was the most trusted device on the network. The printer has since been rated again. Detailed results will be reported separately.

11. Security Considerations

This entire document is a security consideration. Several points nonetheless deserve mention.

Honesty of MITMs. The mechanism assumes that MITMs compute ratings honestly. A dishonest MITM might write arbitrary values. Since a dishonest MITM is Evil, its own traffic will be rated accordingly by the other MITMs on its paths, and it will be unable to report to the ERA, fetch multipliers, or receive Detection Orders. The system is thus self-correcting in the limit, if the limit exists.

Denial of service. An attacker able to write 255 into the Evil Byte of a victim's packets can isolate the victim from every compliant server and client on the Internet. This is indistinguishable from the mechanism operating as intended, and the Working Group therefore does not regard it as an attack.

Integrity. There is no authentication of the octet. Adding one would require a signature, which requires more bits, and the octet has no more bits. See Section 3.2 regarding the Flow Label.

Rating manipulation. An Autonomous System might lower its rating by deliberately losing matches, which it does by sending Good traffic to Evil servers. Since sending Good traffic is the intended behaviour, the Working Group regards this attack as the mechanism working. Conversely, an AS might raise a rival's rating by sending Evil traffic from the rival's address space. This is spoofing, which is Evil, and will be rated as such by any MITM that has read Section 4.3 and knows what an address is for.

Centralisation. The ERA is a single point of failure and a single point of control. See Section 6.7, which the Working Group considers to have dealt with the matter.

Key disclosure. Voluntary Decryption Assistance (Section 4.5.3) transmits session keys in the clear to an intermediary. The Working Group notes that this is what the mechanism is for, and that the mechanism is Voluntary, Trusted, and Proportionate, all of which are words.

Circumvention. An endpoint might attempt to evade rating by tunnelling, which is rated 1.5 (Section 4.4); by using a transport the MITM does not recognise, which is rated 2.0; by not sending packets, which is Good and is RECOMMENDED; or by avian carrier, which the falconer will handle.

12. Privacy Considerations

This document has no privacy considerations, in the sense that the Working Group did not consider privacy. Readers who would like to consider it are referred to Section 4.5, after which they will not need to.

13. IANA Considerations

This document makes the following requests of IANA, which IANA is encouraged to grant before it is rated.

Differentiated Services Field Codepoints. IANA is requested to record, against every codepoint in the DSCP registry, the value “EVIL (see draft-traviss-evil-byte)”. IANA is further requested to record the same value against the ECN field, which has no registry, in whatever it has instead.

HTTP Status Codes. IANA is requested to create the 6xx class in the HTTP Status Code Registry and to register 666, Evil, with this document as reference. IANA is requested not to register any other 6xx code on behalf of anyone else. The Working Group does not wish to share.

HTTP/2 and HTTP/3 Error Codes. IANA is requested to register the error code EVIL with the value ox666 in both registries.

HTTP Field Names. IANA is requested to register the fields Evil and Evil-Threshold, both permanent, both with this document as reference, and both with the status “Evil”.

Well-Known URIs. IANA is requested to register the well-known URI suffix “evil” (Section 7.3).

Special-Use Domain Names. IANA is requested to enter evil.arpa in the Special-Use Domain Names registry [RFC6761], and to delegate evil.arpa to the ERA once the ERA has determined who it is (Section 6.6). The Working Group notes that .arpa [RFC3172] has become the domain in which the Internet keeps the things it would rather not discuss, and that this is a natural fit.

Service Names and Port Numbers. IANA is requested to register the service name esrp on UDP port 666, alongside doom. The Working Group has consulted the existing assignee and received no objection, or indeed any response.

TLS ExtensionType Values. IANA is requested to assign the value 1638 (ox0666) to the extension evil_key_share (Appendix D), for aesthetic reasons.

IPv6 Hop-by-Hop Options. IANA is requested to assign an option type for the Evil Option of Appendix E with the "act" bits set to 00 (skip over) and the "chg" bit set to 1, the value changing en route. The Working Group observes that this will be a rare option whose bits accurately describe its behaviour, and that routers will drop it anyway.

RFC 3514. IANA is requested to annotate the reserved bit of the IPv4 Flags field as "Obsoleted; see Section 3.5", and to leave it exactly where it is.

---

14. References

14.1. Normative References

[RFC791] Postel, J., "Internet Protocol", STD 5, RFC 791, September 1981.

[RFC1035] Mockapetris, P., "Domain names - implementation and specification", STD 13, RFC 1035, November 1987.

[RFC1149] Waitzman, D., "Standard for the transmission of IP datagrams on avian carriers", RFC 1149, 1 April 1990.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.

[RFC2324] Masinter, L., "Hyper Text Coffee Pot Control Protocol (HTCPCP/1.0)", RFC 2324, 1 April 1998.

[RFC2474] Nichols, K., Blake, S., Baker, F., and D. Black, "Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers", RFC 2474, December 1998.

[RFC3168] Ramakrishnan, K., Floyd, S., and D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP", RFC 3168, September 2001.

[RFC3514] Bellovin, S., "The Security Flag in the IPv4 Header", RFC 3514, 1 April 2003.

[RFC6214] Carpenter, B. and R. Hinden, "Adaptation of RFC 1149 for IPv6", RFC 6214, 1 April 2011.

[RFC6919] Barnes, R., Kent, S., and E. Rescorla, "Further Key Words for Use in RFCs to Indicate Requirement Levels", RFC 6919, 1 April 2013.

[RFC7168] Nazar, I., "The Hyper Text Coffee Pot Control Protocol for Tea Efflux Appliances (HTCPCP-TEA)", RFC 7168, 1 April 2014.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, May 2017.

[RFC8200] Deering, S. and R. Hinden, "Internet Protocol, Version 6 (IPv6) Specification", STD 86, RFC 8200, July 2017.

[RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, December 2017.

[RFC8446] Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, August 2018.

[RFC8615] Nottingham, M., "Well-Known Uniform Resource Identifiers (URIs)", RFC 8615, May 2019.

[RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, June 2022.

[ELO] Elo, A. E., "The Rating of Chessplayers, Past and Present", Arco Publishing, 1978.

14.2. Informative References

[RFC1349] Almquist, P., “Type of Service in the Internet Protocol Suite”, RFC 1349, July 1992.

[RFC1591] Postel, J., “Domain Name System Structure and Delegation”, RFC 1591, March 1994.

[RFC1925] Callon, R., “The Twelve Networking Truths”, RFC 1925, 1 April 1996.

[RFC2549] Waitzman, D., “IP over Avian Carriers with Quality of Service”, RFC 2549, 1 April 1999.

[RFC3172] Huston, G., Ed., “Management Guidelines & Operational Requirements for the Address and Routing Parameter Area Domain (“arpa”)”, BCP 52, RFC 3172, September 2001.

[RFC3849] Huston, G., Lord, A., and P. Smith, “IPv6 Address Prefix Reserved for Documentation”, RFC 3849, July 2004.

[RFC4271] Rekhter, Y., Ed., Li, T., Ed., and S. Hares, Ed., “A Border Gateway Protocol 4 (BGP-4)”, RFC 4271, January 2006.

[RFC5398] Huston, G., “Autonomous System (AS) Number Reservation for Documentation Use”, RFC 5398, December 2008.

[RFC5737] Arkko, J., Cotton, M., and L. Vegoda, “IPv4 Address Blocks Reserved for Documentation”, RFC 5737, January 2010.

[RFC6598] Weil, J., Kuarsingh, V., Donley, C., Liljenstolpe, C., and M. Azinger, “IANA-Reserved IPv4 Prefix for Shared Address Space”, BCP 153, RFC 6598, April 2012.

[RFC6761] Cheshire, S. and M. Krochmal, “Special-Use Domain Names”, RFC 6761, February 2013.

[RFC7725] Bray, T., “An HTTP Status Code to Report Legal Obstacles”, RFC 7725, February 2016.

[BLUG] Bergen Linux User Group, “The highly unofficial CPIP WG”, April 2001, https://www.blug.linux.no/rfc1149/.

[CSAR] European Commission, “Proposal for a Regulation of the European Parliament and of the Council laying down rules to prevent and combat child sexual abuse”, COM(2022) 209 final, May 2022.

[OSA] “Online Safety Act 2023”, 2023 c. 50, section 121, United Kingdom, October 2023.

[IANA-PORTS] IANA, “Service Name and Transport Protocol Port Number Registry”, https://www.iana.org/assignments/service-names-port-numbers/.

Appendix A. Reference Implementation

The following Python module implements the formula of Section 4 and the update rule of Section 6. It produces the vectors of Appendix C. Being a Code Component, it is provided without warranty; being this document’s, it is provided without much hope either.

```
"""Reference implementation of draft-traviss-evil-byte-00.

Computes the Evil Rating (ER) carried in the Evil Byte (Section 4) and
the Elo update applied by the Evil Rating Authority (Section 6).
Pure Python 3, no dependencies.  This code is Good (self-assessed;
see Section 5.3).
"""
import math
from datetime import datetime

BASE_EVIL = 16.0           # B: no packet is entirely innocent (Section 4.1)
```

```
# Section 4.1, Table: weights
W = {"as": 1.0, "net": 0.5, "tx": 0.25, "content": 1.5,
     "name": 0.75, "time": 0.25}

# Section 4.3: network protocol factor
NET = {"ipv4": 1.5, "ipv4-cgnat": 1.75, "ipv6": 0.75,
       "ipv6-transition": 1.25, "ipv6-hbh": 2.0}

# Section 4.4: transport factor
TX = {"tcp": 1.0, "udp": 1.1, "quic": 1.25, "sctp": 0.8,
      "icmp-echo": 0.5, "icmp-redirect": 4.0, "tunnel": 1.5,
      "avian": 0.25, "other": 2.0}

# Section 4.5: content factor
CONTENT = {"good": 0.5, "uncertain": 1.5, "evil": 4.0,
           "unanalysed": 2.0, "encrypted": 4.0}
COOPERATION_DISCOUNT = 0.9  # Section 4.5.3

# Section 4.6: nomenclature factor, keyed by top-level label
NAME = {"mil": 4.0, "gov": 2.0, "zip": 2.0, "ai": 1.5, "biz": 1.5,
        "io": 1.25, "com": 1.0, "net": 1.0, "edu": 0.9, "org": 0.8,
        "int": 0.75, "eu": 0.5, "local": 0.5}
NO_NAME = 1.25
PROTEST = ("secure", "trust", "safe", "legit")   # doth protest too much


def name_factor(name):
    """Section 4.6.  `name` is the PTR name, Host, or SNI; None if absent."""
    if not name:
        return NO_NAME
    labels = name.lower().rstrip(".").split(".")
    if labels[-2:] == ["home", "arpa"]:
        f = NAME["local"]                  # it is your printer
    else:
        f = NAME.get(labels[-1], 1.0)      # ccTLDs and unlisted gTLDs: 1.0
    if any(word in name.lower() for word in PROTEST):
        f = max(f, 1.5)
    return f


def time_factor(when, dst=False):
    """Section 4.7.  `when` is a naive datetime in the source's local time."""
    if when.month == 4 and when.day == 1:
        return math.inf                          # all packets are Evil
    if 2 <= when.hour < 5:
        f = 1.5
    elif when.weekday() == 4 and when.hour >= 16:  # Friday afternoon
        f = 1.25
    else:
        f = 1.0
    return f + (0.1 if dst else 0.0)


def content_factor(result, vda=False):
    """Section 4.5.  `result` is a key of CONTENT.  With VDA, `result` is
    the cleartext result and the Cooperation Discount applies."""
    return CONTENT[result] * (COOPERATION_DISCOUNT if vda else 1.0)
```

```
def evil_rating(f_as, f_net, f_tx, f_content, f_name, f_time, arriving=0):
    """Section 4.1.  Returns the ER to write into the octet.
    `arriving` is the value found in the octet on arrival (Section 5.4)."""
    f_tamper = 1.0 if arriving == 0 else 1.5          # Section 4.8
    x = BASE_EVIL * f_tamper
    for key, f in (("as", f_as), ("net", f_net), ("tx", f_tx),
                   ("content", f_content), ("name", f_name),
                   ("time", f_time)):
        x *= f ** W[key]
    if math.isinf(x):
        er = 255                                      # 1 April
    else:
        er = int(math.floor(x + 0.5))                 # round half towards Evil
        er = max(1, min(255, er))
    return max(er, arriving)                          # Evil is monotonic


# ---- Section 6: the Evil Rating Authority --------------------------------

def as_multiplier(rating):
    """Section 6.1: F_AS from an ERA rating."""
    return max(0.25, min(4.0, 2.0 ** ((rating - 1500.0) / 400.0)))


def k_factor(rating, exchanges):
    """Section 6.3.1."""
    if exchanges < 30:
        return 64          # provisional
    if rating >= 2400:
        return 16          # Grandmaster of Evil
    return 32


def expected_score(r_a, r_b):
    """Section 6.3."""
    return 1.0 / (1.0 + 10.0 ** ((r_b - r_a) / 400.0))


def elo_update(r_a, r_b, er_req, er_resp, n_a=30, n_b=30):
    """Sections 6.2 and 6.3.  a is the client's AS, b the server's.
    The more Evil party wins.  Returns the two new ratings."""
    s_a = 1.0 if er_req > er_resp else (0.5 if er_req == er_resp else 0.0)
    e_a = expected_score(r_a, r_b)
    k = min(k_factor(r_a, n_a), k_factor(r_b, n_b))   # Section 6.3.1
    return (r_a + k * (s_a - e_a),
            r_b + k * ((1.0 - s_a) - (1.0 - e_a)))
```

Appendix B. Deployment on Linux

This appendix describes how the author deployed the specification on a small network using nftables and a userspace MITM. Addresses are drawn from the documentation ranges [RFC5737] [RFC3849], which the Working Group notes are the only addresses on the Internet that have never done anything wrong.

B.1. nftables

Since the octet is the Differentiated Services field by another name, ER = (DSCP << 2) | ECN, and ER >= 128 is equivalent to DSCP >= 32. A fixed rating can therefore be written, and a threshold enforced, with stock nftables. The formula itself is computed in userspace: the rules below hand packets from the demonstration subnet to the MITM of Appendix B.2 on queue 666.

```
table inet evil {
    # Section 5: the MITM. Packets from the demonstration subnet are sent
    # to a userspace MITM (Appendix B.2) on queue 666, which rates them.
    chain forward {
        type filter hook forward priority mangle; policy accept;
        ip saddr 192.0.2.0/24 meta l4proto { tcp, udp } queue num 666 bypass
        ip6 saddr 2001:db8::/32 meta l4proto { tcp, udp } queue num 666 bypass
    }
    # A fixed-value MITM, for demonstrations that do not need the formula.
    # DSCP = ER >> 2, ECN = ER & 3.  ER = 0xCA (202): DSCP 0x32, ECN 2.
    chain forward_fixed {
        type filter hook forward priority mangle + 1; policy accept;
        ip saddr 192.0.2.66 ip dscp set 0x32 ip ecn set 2
        ip6 saddr 2001:db8::66 ip6 dscp set 0x32 ip6 ecn set 2
    }
    # Section 7: server-side enforcement below the application layer.
    # ER >= 128 is equivalent to DSCP >= 32 (raw match shown for IPv6).
    chain input {
        type filter hook input priority filter; policy accept;
        ip dscp >= 32 tcp dport 80 counter reject with tcp reset
        @nh,4,8 >= 128 meta nfproto ipv6 tcp dport 80 counter reject with tcp reset
    }
    # Section 8: client-side enforcement — discard responses from Evil servers.
    chain input_client {
        type filter hook input priority filter + 1; policy accept;
        ip dscp >= 32 tcp sport 80 counter drop
        ip6 dscp >= 32 tcp sport 80 counter drop
    }
}
```

B.2. A Minimal MITM

The following program consumes packets from queue 666, rates them, writes the octet, and returns them to the kernel. It performs no content analysis and presumes accordingly (Section 4.5.2), treating anything bound for a port it associates with encryption as Encrypted With Intent. It is illustrative rather than normative. A production MITM would read everything.

```
#!/usr/bin/env python3
"""A minimal MITM (Section 5) for Linux.  Rates every packet that nftables
sends to NFQUEUE 666 (Appendix B.1) and writes the Evil Byte.

Requires the netfilterqueue and scapy packages, root, and a clear
conscience.  Illustrative, not normative: a production MITM would read
everything (Section 4.5.1); this one merely presumes."""
import socket
from datetime import datetime
from netfilterqueue import NetfilterQueue
from scapy.all import IP, IPv6
import evilbyte as eb
```

```
QUEUE = 666
AS_MULTIPLIER = {}        # populated from <asn>.as.evil.arpa (Section 6.5)
DEFAULT_AS = 1.0          # ERA has no opinion of this AS (Section 4.2)
ENCRYPTED_PORTS = {443, 853, 993, 995, 8443}   # Encrypted With Intent


def name_of(addr):
    """PTR lookup (Section 4.6).  None if the source is nameless."""
    try:
        return socket.gethostbyaddr(addr)[0]
    except OSError:
        return None


def rate(raw):
    if raw[0] >> 4 == 4:
        pkt, arriving, f_net = IP(raw), IP(raw).tos, eb.NET["ipv4"]
        proto = pkt.proto
    else:
        pkt, arriving, f_net = IPv6(raw), IPv6(raw).tc, eb.NET["ipv6"]
        proto = pkt.nh
        if proto == 0:                                  # Hop-by-Hop Options
            f_net = eb.NET["ipv6-hbh"]
    f_tx = {6: eb.TX["tcp"], 17: eb.TX["udp"], 132: eb.TX["sctp"],
            1: eb.TX["icmp-echo"], 58: eb.TX["icmp-echo"],
            47: eb.TX["tunnel"], 4: eb.TX["tunnel"],
            41: eb.TX["tunnel"], 50: eb.TX["tunnel"]}.get(proto, eb.TX["other"])
    dport = getattr(pkt.payload, "dport", None)
    if proto == 17 and dport == 443:
        f_tx = eb.TX["quic"]
    f_content = eb.content_factor("encrypted" if dport in ENCRYPTED_PORTS
                                  else "unanalysed")   # analysis is OPTIONAL
    er = eb.evil_rating(AS_MULTIPLIER.get(pkt.src, DEFAULT_AS), f_net, f_tx,
                        f_content, eb.name_factor(name_of(pkt.src)),
                        eb.time_factor(datetime.now()), arriving)
    if isinstance(pkt, IP):
        pkt.tos = er
        del pkt.chksum                                  # recomputed on send
        pkt.flags = (int(pkt.flags) & 3) | (4 if er >= 128 else 0)   # RFC 3514
    else:
        pkt.tc = er
    return bytes(pkt)


def handle(packet):
    packet.set_payload(rate(packet.get_payload()))
    packet.accept()


if __name__ == "__main__":
    nfq = NetfilterQueue()
    nfq.bind(QUEUE, handle)
    try:
        nfq.run()
    finally:
        nfq.unbind()
```

B.3. Reading the Octet at the Application Layer

For datagram sockets, the operating system will surface the octet of each received datagram on request: on Linux, the IP_RECVTOS and IPV6_RECVTCLASS socket options cause it to be delivered as ancillary data with recvmsg().

For stream sockets, the kernel does not surface the octet of received segments to the application. An Evil-aware host firewall, which may be a second instance of the program in Appendix B.2 attached to the input hook, records the greatest ER observed for each (source address, source port) pair in a table, the Evil Table, which the application consults on accepting a connection. From it the application sets the Evil field of Section 7.4 and chooses between serving the request and returning 666. Entries SHOULD be removed when the connection closes and MUST NOT be removed before. Evil, once observed, is not forgotten until the socket is.

Appendix C. Test Vectors

The following vectors were produced by the reference implementation of Appendix A, at noon on Wednesday 31 March 2027 unless otherwise stated, from an AS that the ERA has not rated unless otherwise stated. Implementations MUST agree with them, and MAY be surprised by them.

ID	SCENARIO	F_AS	F_NET	F_TX	F_CONTENT	F_NAME	F_TIME	ARRIVING	ER
C.1	Baseline: IPv4, TCP, .com, not analysed, Wednesday noon	1	1.5	1	2	1	1	0	**55**
C.2	As C.1, but over TLS (Encrypted With Intent)	1	1.5	1	4	1	1	0	**157**
C.3	As C.2, but over IPv6	1	0.75	1	4	1	1	0	**111**
C.4	As C.2, with VDA; plaintext found Good	1	1.5	1	0.45	1	1	0	**6**
C.5	As C.2, with VDA; plaintext found Evil	1	1.5	1	3.6	1	1	0	**134**
C.6	AS32934, QUIC over IPv4, encrypted, 03:00	2	1.5	1.25	4	1	1.5	0	**255**
C.7	EU institution, IPv6, TCP, analysed Good, .eu	0.5	0.75	1	0.5	0.5	1	0	**1**
C.8	.mil, IPv4, TCP, encrypted	4	1.5	1	4	4	1	0	**255**
C.9	Avian carrier, IPv4, scroll not unrolled, .org	1	1.5	0.25	2	0.8	1	0	**33**
C.10	Printer: mDNS (UDP) over IPv4, analysed Good, .local	1	1.5	1.1	0.5	0.5	1	0	**4**
C.11	As C.1, arriving at a second MITM already rated 55	1	1.5	1	2	1	1	55	**83**
C.12	As C.1, no name, Friday 17:00	1	1.5	1	2	1.25	1.25	0	**69**
C.13	As C.1, from a host called secure-gw.example.net	1	1.5	1	2	1.5	1	0	**75**
C.14	IPv6 with Hop-by-Hop Options, ICMPv6 echo, analysed Good	1	2	0.5	0.5	1	1	0	**7**
C.15	As C.7, on 1 April	0.5	0.75	1	0.5	0.5	inf	0	**255**

The following vectors exercise the update rule of Section 6.3, with K = 32 throughout.

ID	R_A	R_B	ER_REQ	ER_RESP	S_A	E_A	NEW R_A	NEW R_B
C.16	1500	1500	157	17	1	0.5	1516	1484
C.17	1900	1100	157	17	1	0.99	1900.32	1099.68
C.18	1900	1100	17	157	0	0.99	1868.32	1131.68
C.19	1500	1500	55	55	0.5	0.5	1500	1500

C.16 shows a first match between two unrated ASes; the client, being the more Evil, gains sixteen points and F_AS = 1.028. C.17 shows that a Grandmaster-adjacent AS beating an EU AS gains almost nothing, the result having been expected. C.18 shows the upset. C.19 shows a draw, and is included because the Working Group is fond of it.

Appendix D. The evil_key_share TLS Extension

Voluntary Decryption Assistance (Section 4.5.3) is signalled in TLS [RFC8446] by the extension evil_key_share, ExtensionType 1638 (Section 13), whose extension_data is:

```
enum { key_disclosure(1), self_assessment(2), abstinence(3) } VDAForm;

struct {
    VDAForm  form;
    opaque   keys<0..2^16-1>;     /* form 1: the session keys, in the
                                     clear, for efficiency            */
    uint8    assessment;          /* form 2: 0 = Good, 1 = Uncertain,
                                     2 = Evil.  Honesty is expected.  */
} EvilKeyShare;
```

A client sends the extension in its ClientHello. Since no session keys exist at that point, a client using form 1 MUST include the keys it intends to derive, which requires it to know the server's key share in advance. Clients MAY guess.

A server sends the extension in EncryptedExtensions, which the MITM cannot read until the server has assisted. This is the Evil Bootstrap Problem (Section 8.2) again, and is left for the same future work.

Form 3 MAY alternatively be signalled by not sending a ClientHello at all, which is also the most widely deployed form.

Appendix E. Alternative Encodings

Purists who object to the reuse of the Differentiated Services field MAY instead carry the Evil Rating in an IPv4 option or an IPv6 Hop-by-Hop option, as follows.

```
IPv4 Evil Option                IPv6 Evil Option (Hop-by-Hop)
+--------+--------+--------+    +--------+--------+--------+
|  Type  | Len=3  |   ER   |    |  Type  | Len=1  |   ER   |
+--------+--------+--------+    +--------+--------+--------+
Type: copied=1, class=0,        Type: act=00, chg=1,
      number=TBD (Section 13)         number=TBD (Section 13)
```

The IPv4 option is copied on fragmentation so that every fragment carries its Evil (Section 5.5). The IPv6 option is marked as changing en route, which it does.

Purists should be aware that IPv4 options are dropped by a large fraction of the Internet, that Hop-by-Hop options are dropped by most of the rest, and that a packet which is dropped has, per Section 5.4, achieved the only reduction in Evil this specification allows. The Working Group therefore regards the alternative encodings as compliant, effective, and unusable.

Acknowledgements

The author thanks Steven Bellovin for the original bit, and the many policymakers whose proposals made this one look reasonable. Thanks are also due to the Working Group's colleague (Section 4.5.1), whose face has been invaluable.

No pigeons were harmed in the preparation of this document. One was rated.

Author's Address

Rose Traviss
Data Torturing Solutions Ltd
Bristol
United Kingdom
Email: TBD